

DSA STUDY BLUEPRINT SERIES

63. Stock Span Problem Optimal Solution

Calculating Stock Price Span using Monotonic Stacks

Section 08 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Span:** Consecutive days preceding current day where price is less or equal.
- Keep a decreasing stack of pairs: `(price, span)`.

Memory & Execution Blueprint

Prices: [100, 80, 60, 70] $\Rightarrow$ Span of 70 is **2** (since 70 > 60)

C++ Implementation Reference

Standard code layout and syntax

```
stack> st; // price, span
int nextDay(int price) {
    int span = 1;
    while (!st.empty() && st.top().first <= price) {
        span += st.top().second; st.pop();
    }
    st.push({price, span});
    return span;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(1)$ amortized / $O(N)$ total
Space Complexity	$O(N)$

Key Practices & Gotchas

- **Important:** Each price element is pushed and popped at most once.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace monotonic stack modifications for next greater element on `[4, 5, 2]` :

Active Element	Stack State (Indices)	Pop comparison	Result output
4	[0] (value 4)	N/A	Waiting for next greater
5	[1] (value 5)	$5 > 4$ (pop index 0)	Next Greater of 4 is 5

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Monotonic properties:** Stacks process elements strictly increasing or decreasing, optimizing nested lookups.
- **Min Stack details:** Track current minimum bounds by storing elements as encoded difference values.
- **Related LeetCode Problems:** #155 Min Stack, #84 Largest Rectangle in Histogram, #42 Trapping Rain Water.